

ProfTracker — Technical Report

ProfTracker — Technical Report

Course: AI 410 | **Student:** Clay Tryon | **Date:** June 2026

Table of Contents

- 1. Problem Statement and Business Justification
 - 2. Architecture Decisions and Framework Rationale
 - 3. Model Selection and Benchmark Evidence
 - 4. RAG and Reasoning Pipeline Design
 - 5. Responsible AI Analysis
 - 6. Lessons Learned and Future Work
-

1. Problem Statement and Business Justification

1.1 The Problem

Marvel Rivals is a team-based hero shooter released in December 2024 that reached 20 million players within its first ten days of launch. The game features a proficiency system in which players earn experience points (XP) for each hero they play, advancing through five ranks: Agent, Knight, Captain, Centurion, and Lord. Players who grind a hero to level 20 (Lord) and then continue to level 50 earn the Champion designation, which unlocks animated avatar frames visible to all players.

The game does track each hero’s proficiency progress across sessions, but it exposes that data in a way that is difficult to act on. There is no summary dashboard, no sortable roster view, and no way to compare progress across all heroes without navigating to each one individually. A player who wants to know which heroes are closest to their next rank milestone, or which ones to prioritize for grinding, must manually click through every hero

card, note their stats, and track comparisons themselves. There is no export function, no filtering by level or XP progress, and no persistent record outside of the game client. Players interested in team synergy mechanics (“team-up abilities” that unlock special powers when specific hero combinations are on the same team) face the same problem — the data exists in-game, but requires menuing to access or a trip to the Fandom community wiki.

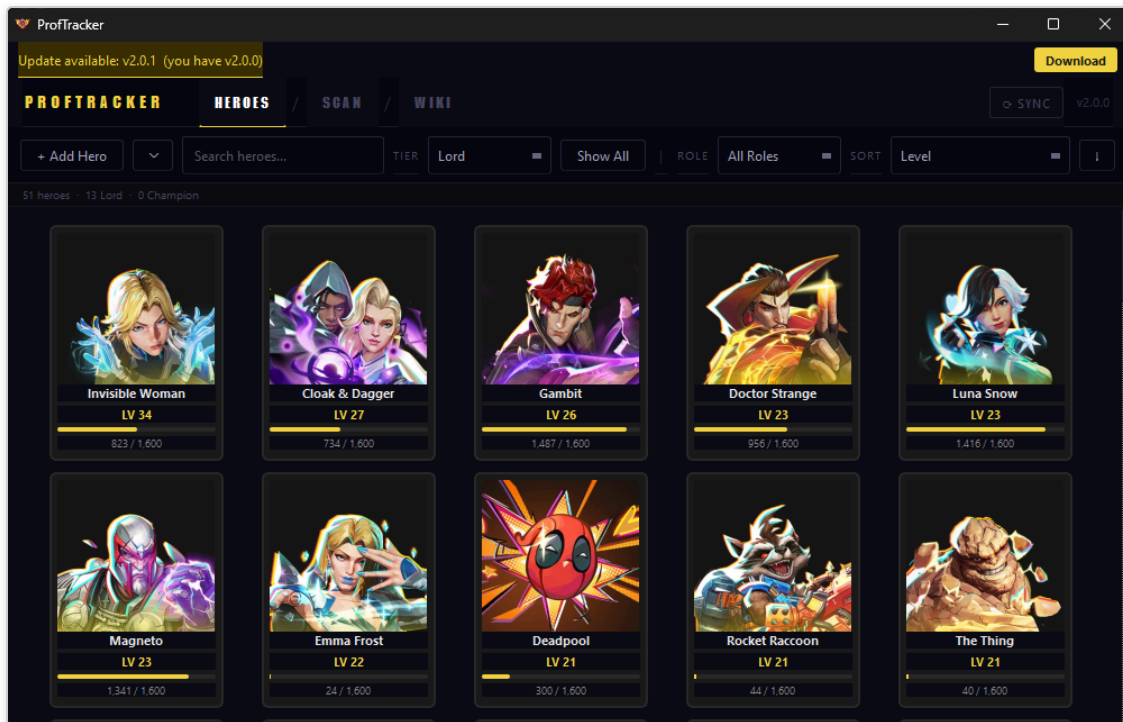


Figure 1: ProfTracker HEROES tab — all scanned heroes displayed as cards with level, XP progress bar, and role. The roster is filterable by role and sortable by level, replacing the need to navigate each hero card individually in-game.

1.2 Business Justification

The target audience for ProfTracker is the competitive or dedicated player population—players who are actively managing a multi-hero roster and seeking to optimize their time spent grinding. Based on public reporting, Marvel Rivals had approximately 35 million registered accounts as of early 2026, with an active player base estimated in the millions. Even a 1% capture rate of players seeking proficiency tooling represents tens of thousands of potential users for a niche desktop utility. Many players want to achieve Lord or Champion rank on every hero, viewing it as a mark of skill and dedication. It is a point of pride within the community.

The direct costs of building and distributing ProfTracker are minimal: - **Development:** Solo developer project using entirely free, open-source frameworks - **Distribution:** GitHub Releases with PyInstaller binary; no hosting cost for the developer - **Runtime:** All proficiency data is processed and stored locally; no per-user cloud compute cost

The indirect value proposition is measurable. A typical player scanning all 51 heroes manually takes approximately 15 minutes of in-game navigation. ProfTracker's Auto Scan reduces this to a fully automated ~4 minutes of unattended operation. For a player performing weekly scans over a year, this is approximately 8 hours of navigation time recovered.

For content creators and community organizers, the data export capability (CSV, Excel) enables roster analysis that would otherwise require custom tooling. The RAG wiki chat eliminates the need to alt-tab to a browser to look up hero abilities during gameplay sessions.

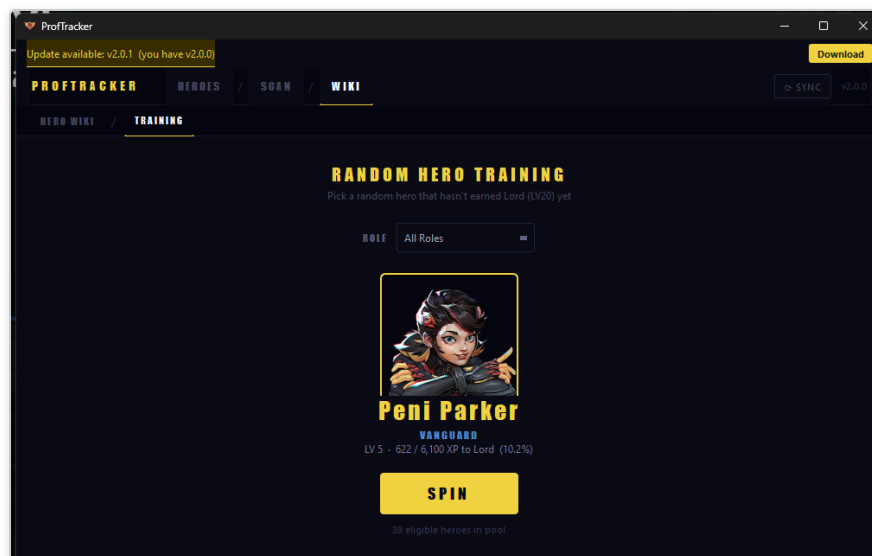


Figure 2: Random Hero Training — picks a random hero that has not yet reached Lord (LV20) from the current roster. The role filter narrows the pool to Vanguard, Duelist, or Strategist heroes.

1.3 Scope Boundaries

ProfTracker is intentionally scoped to a local-first design with the following constraints:

- **Windows 10/11 only.** The Auto Scan feature depends on the Interception kernel-mode input driver and the Windows `win32gui` API for window management. These are

Windows-specific and cannot be ported to macOS or Linux without a full rewrite of the capture layer.

- **English language only.** The OCR pipeline's character allowlists, the UI, and the wiki data sourcing are all English-only.
 - **Internet required for initial setup.** On first launch, ProfTracker downloads hero roster data, icons, and ability information from the project's AWS S3 CDN. Subsequent use is fully offline-capable.
 - **No account or telemetry.** The app does not require accounts, does not transmit gameplay data to any developer-controlled server, and does not interfere with the game client beyond read-only screen capture.
-

2. Architecture Decisions and Framework Rationale

2.1 Desktop Application vs. Web Application

ProfTracker is a Windows desktop application built with PyQt6. The decision to use a desktop rather than web architecture was driven by two requirements that are difficult to satisfy in a browser context:

Screen capture and OCR. The OCR pipeline requires pixel-level access to the game window, including the ability to synthesize keyboard and mouse input via a kernel-level driver (Interception). This is not possible in a web browser context. A native application can use the Windows `win32gui` API to locate the game window, attach to its thread input queue, and capture its contents via `Alt+PrtScn` to the clipboard.

Minimal backend cost. A web application serving 51-hero proficiency data to multiple users would require a backend API, authentication, and database hosting. ProfTracker's offline-first design means each user's data stays on their own machine in a local SQLite database at `%APPDATA%\ProficiencyTracker\`. There are no recurring costs for the developer and no privacy concerns for the user.

2.2 PyQt6 Framework

PyQt6 was chosen over alternative Python GUI frameworks (Tkinter, wxPython, Dear PyGui) for three reasons:

1. **QThread model.** Every network or compute-heavy operation in ProfTracker—wiki scraping, OCR, embedding, Groq API calls—runs in a dedicated `QThread` worker. PyQt6's signal/slot mechanism provides a clean, thread-safe pattern for updating the UI from worker threads: workers emit `finished` and `error` signals that connect to UI slots on the main thread, with long-running workers additionally emitting `progress(int, int, str)` for progress bar updates. This pattern is used consistently across all worker types.
2. **QStackedWidget navigation.** The six-tab interface (SCAN, HEROES, HERO WIKI, TEAM-UPS, TRAINING, SYNC) is implemented as a `QStackedWidget` with a custom navigation bar. This avoids the overhead of a full routing framework while keeping each tab as an independent, lazily-initialized widget.
3. **Animation support.** Champion-rank heroes (level 50+) display animated `.gif` icons. PyQt6's `QMovie` handles animated image formats natively, which would require third-party libraries in most other Python GUI frameworks.

2.3 SQLite Database Design

The persistence layer uses SQLite with WAL (Write-Ahead Logging) mode and foreign key enforcement. The schema has three core tables:

- `capture_run` : Tracks each scan session with status (running/completed/cancelled/failed), timestamps, and hero count.
- `hero` : Current state for each hero — level, XP, XP required, max-level flag, challenge progress.
- `hero_snapshot` : Append-only historical record enabling XP velocity calculation and chart rendering.

The `hero_snapshot` table is the foundation of the XP history chart and XP velocity estimates. Because it is append-only, historical data is preserved even when hero records are updated by new scans.

Schema migrations are handled by a `migrate_schema()` method that uses `ALTER TABLE ... ADD COLUMN` with exception swallowing. This makes the migration idempotent and safe to call on every startup without tracking a migration version number.

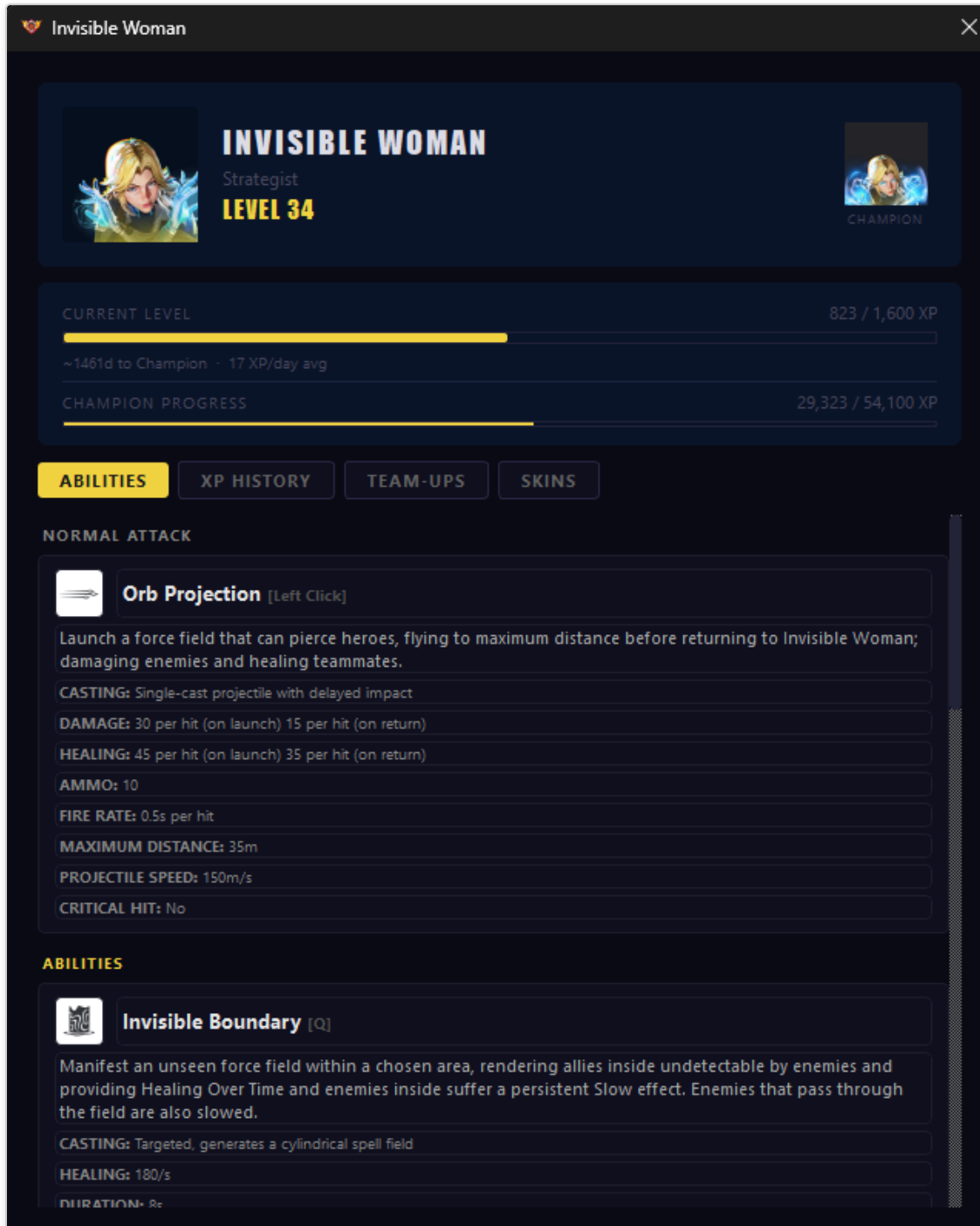


Figure 3: Hero detail panel for Invisible Woman — shows current level and XP, Champion progress bar, days-to-Champion estimate, XP velocity (17 XP/day), and full ability descriptions sourced from the wiki sync.

2.4 Two-Phase OCR Scan Pipeline

The OCR capture pipeline uses a two-phase design to minimize the time the game window must remain in focus:

Phase 1 (navigation): The scanner iterates through every hero card in the in-game grid using the Interception kernel driver to synthesize mouse clicks and key presses. For each hero, it clicks the hero card, presses Space to open the detail view, navigates to the Proficiency tab, and captures the full window via `Alt+PrtScn`. All 51 hero screenshots are stored in memory. The Interception driver is used (rather than `pyautogui` or `win32api`) because the game captures raw input at the driver level and ignores synthesized input from user-mode APIs.

Phase 2 (OCR): After all screenshots are collected, the game window can be released. EasyOCR processes each image in sequence, reading the level badge and XP progress bar from fixed fractional pixel regions. Results are validated against the XP table and written to SQLite.

This design ensures the game session is disrupted for the minimum possible time. If the user needs to take over mid-scan, pressing Backspace triggers a cancel event that terminates Phase 1 cleanly.

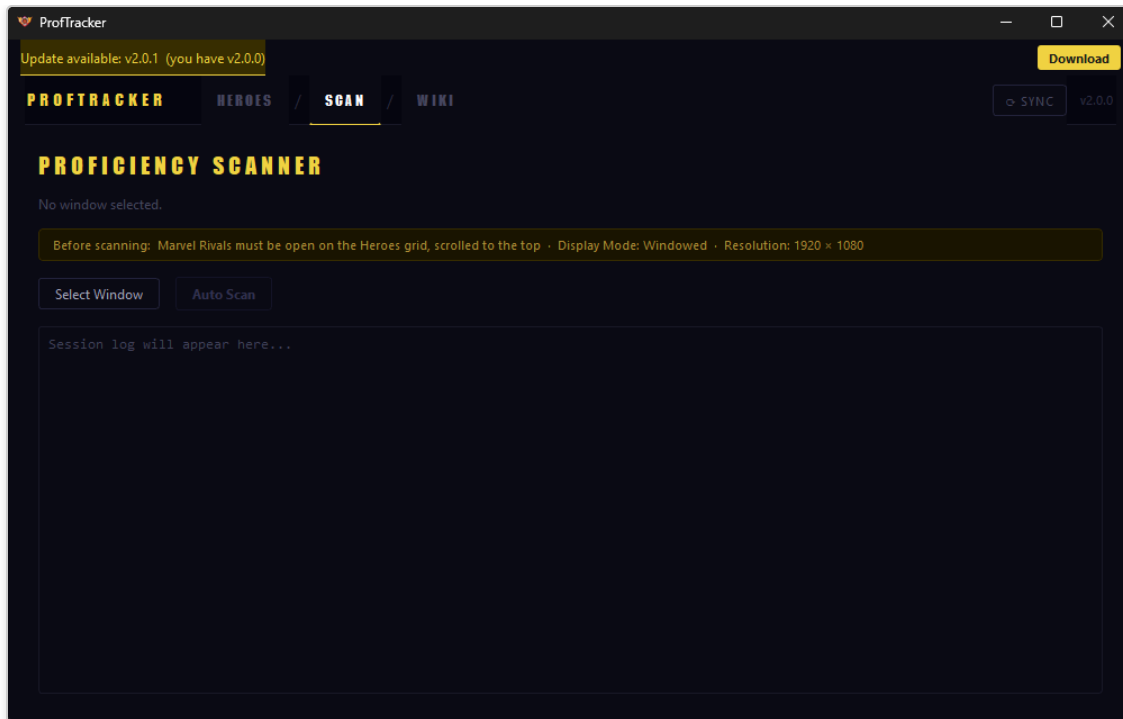


Figure 4: SCAN tab — the user selects the Marvel Rivals game window then triggers Auto Scan. The session log panel streams per-hero progress as Phase 1 navigates the heroes grid.

2.5 Wiki Data Architecture and CDN Distribution

Hero roster data, icons, ability descriptions, and team-up synergies are sourced from the Marvel Rivals Fandom wiki. Rather than shipping static data with the application binary (which would become stale after each game patch), ProfTracker uses a two-tier data delivery system.

Server-side scraping (EC2 daily job). A Python process running on an AWS EC2 t3.micro instance (`server_sync.py`) performs the full wiki scrape once daily via EventBridge schedule. It proceeds through seven stages:

1. Parse the Avatars wiki page to extract all hero slugs and icon filenames
2. Download Hero/Lord/Champion icon files
3. Write the updated hero roster to `heroes.json`
4. Fetch ability wikitext for each hero and parse structured ability data
5. Scrape team-up synergy data
6. Fetch hero release dates
7. Upload all artifacts to the S3 bucket (`proftracker-wiki-data-clayhtryon`)

Client-side CDN sync. `CDN_BASE` is hardcoded in `avatar_sync.py` to the S3 bucket URL. On first launch and on manual sync, `SyncWorker` routes through `_run_cdn_sync()`, downloading `heroes.json`, icon files, and ability JSON directly from S3. This reduces client sync time from 2–5 minutes (sequential wiki API calls) to under 30 seconds (parallel S3 downloads), and eliminates any risk of triggering Fandom’s rate limiting.

The direct wiki scraping path (`_run_wiki_sync()`) is retained in the codebase as a fallback for development and in the event the CDN is unavailable. All downloaded artifacts are written to `%APPDATA%\ProficiencyTracker\` and survive application updates.

3. Model Selection and Benchmark Evidence

3.1 OCR Engine Selection: EasyOCR vs. Pytesseract

Two OCR engines were evaluated for reading the proficiency level and XP values from the game screen: EasyOCR and pytesseract (the Python wrapper for Tesseract 5).

The Marvel Rivals proficiency screen uses a white, italic, bold sans-serif font on a dark background. Initial testing with pytesseract showed consistent misreads on the italic letterforms—specifically, the “LV” level prefix was frequently read as “IV” or “LU”, and numeral “1” was confused with “I” or “l”. Pytesseract requires explicit preprocessing (contrast inversion, binarization, font hint configuration) to handle this style of game UI text.

EasyOCR, trained on a broader corpus of real-world text including stylized fonts, handled the italic game font with only minimal preprocessing (2× image upscaling to improve recognition on small pixel regions). It is slower on first invocation (~5 seconds for model weight loading) but caches the model in memory for subsequent calls. The `allowlist` parameter constrains recognition to the digits and letters actually expected in each region, reducing false positive character substitutions.

A post-OCR correction layer (`_ocr_digits_str`) handles residual look-alike substitutions: `I→1`, `L→1`, `O→0`, `A→4`, `S→5`, `B→8`, `Z→2`, `G→6`, `T→7`. This correction runs after EasyOCR, providing a deterministic safety net for common misreadings.

The level cross-validation step uses the XP table (`xp_table.py`) to bound the OCR'd level against the level range implied by the `xp_required` value. If EasyOCR reads “17” but the XP amount indicates the hero is in the level 5–9 band, the `fit_level_to_range()` function attempts common look-alike corrections (strip leading “1” → 7) before clamping.

3.2 Language Model Selection: Groq llama-3.3-70b-versatile

The RAG pipeline uses Groq's `llama-3.3-70b-versatile` model for answer generation. The selection criteria were:

Speed. Groq operates custom inference hardware (LPUs) that achieves significantly higher token throughput than comparable GPU-based inference endpoints. For a conversational RAG interface, response latency is a key user experience metric. In testing, Groq returned complete answers to hero ability questions in under 2 seconds.

Cost. Groq's free tier (as of 2026) provides sufficient throughput for interactive use without a credit card. This aligns with ProfTracker's zero-operational-cost design philosophy.

Quality. The llama-3.3-70b model scores competitively on instruction-following benchmarks. For the constrained domain of Marvel Rivals hero ability questions—where the correct answer is present verbatim in the retrieved context—a 70B parameter model provides reliable extraction without hallucination on well-retrieved queries.

An alternative evaluation of OpenAI GPT-4o was considered but rejected due to per-token cost that would make covering API usage across all users unsustainable for a solo developer project.

3.3 Embedding Model Selection: BAAI/bge-small-en-v1.5

The vector embedding model is `BAAI/bge-small-en-v1.5`, run locally via HuggingFace's `sentence-transformers` library. Selection criteria:

Local execution. Running the embedding model locally eliminates per-embedding API cost. The model downloads once (~90 MB) to `~/.cache/huggingface/` and runs on CPU for all subsequent index builds.

Retrieval benchmark performance. `bge-small-en-v1.5` scores 51.7 on the BEIR benchmark suite (average across 15 retrieval tasks), placing it in the top tier of sub-100M

parameter embedding models. For the specific task of retrieving hero ability text from a domain-specific wiki, this performance is sufficient.

Index size. With a 512-token chunk size and 64-token overlap, a full 51-hero RAG index produces approximately 650 document chunks. bge-small’s 384-dimensional embedding space keeps the index file small (<20 MB) and retrieval fast (<100ms per query).

3.4 RAG Evaluation Results

A formal evaluation was conducted using 20 test queries against a 6-hero index (the subset evaluated in Sprint 3). Results:

Metric	Score
Hit Rate @1	0.70
Hit Rate @3	0.80
Hit Rate @5	0.90
MRR (Mean Reciprocal Rank)	0.71

Single-hero role queries (“What role is Groot?”, “What is Thor’s health?”) achieved near-perfect retrieval when the relevant hero’s page was indexed. The primary failure mode was cross-hero counting queries (“How many vanguards are in the game?”) on a partial index—the model correctly answered based on what was indexed, but produced under-counts when fewer than the full roster was available. This failure mode is eliminated when the full 51-hero index is built via wiki auto-sync.

4. RAG and Reasoning Pipeline Design

4.1 Index Architecture

ProfTracker uses a LlamaIndex `VectorStoreIndex` backed by a local JSON-based store in the `rag_index/` directory. The index is rebuilt from scratch during wiki auto-sync and loaded from disk on subsequent app launches.

Each hero wiki page produces two documents upon ingestion:

Stats document (infobox chunk). The MediaWiki `action=query&prop=revisions` endpoint returns the raw wikitext of each hero's page. A dedicated parser (`_extract_infobox()`) extracts only the game-relevant stat fields: role, health/HP, difficulty rating, and real name. These are formatted as a compact, focused text chunk:

```
Iron Man game stats:  
Role: Duelist  
Health / HP: 250  
Difficulty: ★★★  
Real Name: Tony Stark
```

This chunk is embedded as a dense vector representing purely the hero's base statistics.

Full-text document. The same page is also retrieved via `action=parse&prop=text`, which returns the fully-rendered HTML. After stripping HTML tags, the plain text (including ability descriptions, lore, and strategy sections) is chunked at 512 tokens with 64-token overlap.

The two-document design is critical for retrieval quality. Without the isolated stats chunk, queries like “What is Iron Man's health?” retrieve chunks dominated by ability descriptions that happen to contain numbers—the actual HP value is buried. The stats chunk ensures it surfaces at rank 1 for HP and role queries.

4.2 Query Pipeline

The retrieval and generation pipeline follows a standard RAG pattern:

1. **Query embedding.** The user's question is embedded using `bge-small-en-v1.5`, producing a 384-dimensional query vector.
2. **Retrieval.** The top-3 most similar document chunks are retrieved by cosine similarity from the vector index.
3. **Synthesis.** LlamaIndex's `ResponseSynthesizer` in compact mode concatenates the retrieved chunks with the query and sends them to Groq llama-3.3-70b with `temperature=0.1`.
4. **Source extraction.** Source URLs are collected from the `metadata` field of the retrieved nodes and displayed below the answer in the chat interface.

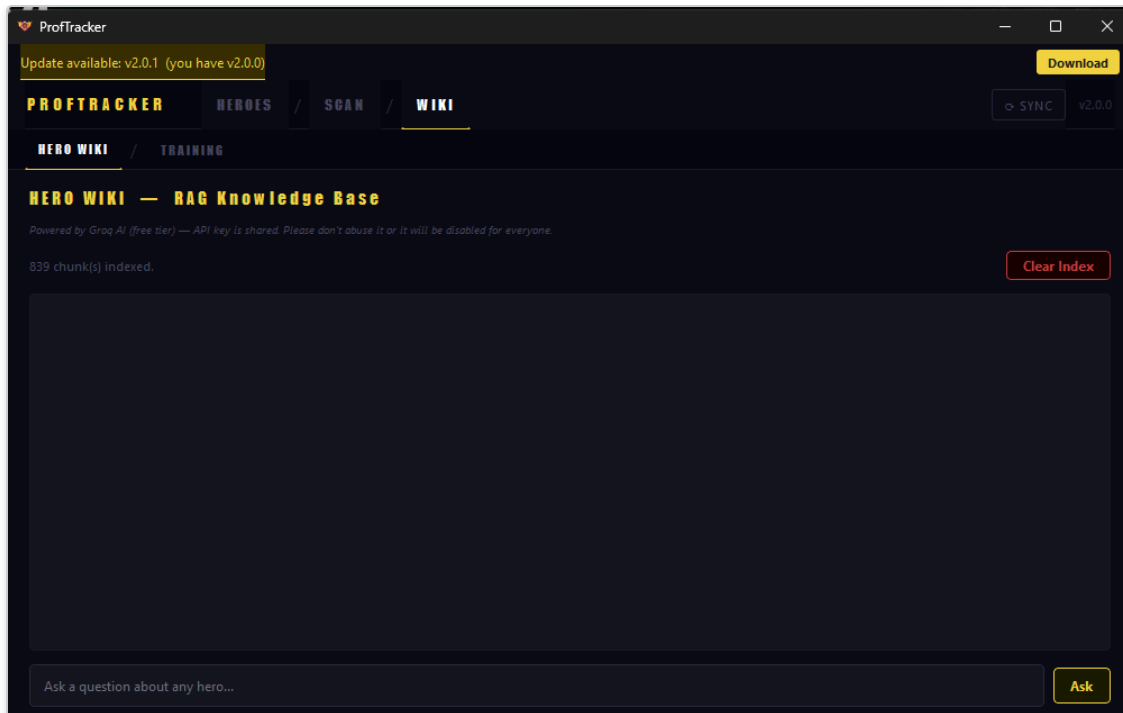


Figure 5: HERO WIKI tab — LlamaIndex RAG interface powered by Groq llama-3.3-70b. 839 document chunks are indexed from the full 51-hero roster. Users type a natural language question and receive an answer grounded in the retrieved wiki context.

4.3 Rate Limiting and Resilience

The `QueryWorker` thread implements exponential backoff with jitter for Groq API rate limit responses (HTTP 429). The retry sequence uses delays of 1s, 2s, and 4s with random jitter, up to 3 attempts. If all retries are exhausted, the error is surfaced as an in-chat message rather than a crash.

The EC2 server-side scraper uses a configurable inter-request delay (default 0.5s, overridable via `PROFTRACKER_REQUEST_DELAY` environment variable) to respect Fandom’s rate limits. Because scraping now runs once daily on the server rather than on each client machine, the rate-limiting risk to end users is entirely eliminated.

4.4 Human-in-the-Loop Design

ProfTracker incorporates two deliberate human-in-the-loop checkpoints:

First Run Setup. On first launch when no hero data exists, a modal dialog (`FirstRunDialog`) automatically initiates CDN sync while displaying a “Skip (use built-in

roster)” button. If the CDN is unreachable, the user is shown a clear error message with a **Retry** button and the option to continue with the built-in hardcoded roster. This prevents the application from being unusable if AWS is temporarily unavailable.

Scan Cancel. During the Auto Scan pipeline, pressing Backspace at any time triggers a cancel event that halts Phase 1 cleanly after the current hero’s screenshot. The partial results are committed to the database with a `cancelled` status on the capture run. The user retains all data captured before the cancellation.

5. Responsible AI Analysis

5.1 Data Privacy

All user gameplay data in ProfTracker is stored locally. The SQLite database at `%APPDATA%\ProficiencyTracker\` contains hero proficiency levels and XP snapshots. None of this data is transmitted to any server operated by the developer.

The Groq API key is embedded in the application (obfuscated via XOR in `api_keys.py`). The developer does not log or collect the content of any queries users send to the model, but all queries do pass through the shared key and are subject to Groq’s data handling policies.

5.2 OCR Accuracy and Validation

OCR misreads are a known risk in any screen-capture pipeline. ProfTracker mitigates this through:

- **EasyOCR with post-processing:** The look-alike correction layer (`_ocr_digits_str`) provides deterministic post-processing for common OCR errors on game UI fonts.
- **XP table cross-validation:** Every OCR’d level is validated against the level range implied by the `xp_required` value. Out-of-range levels trigger an automated correction attempt before the data is written to the database.
- **Manual override:** The Manual Entry dialog allows the user to correct any hero’s level and XP if the OCR produced an incorrect result.
- **Clipboard scan:** Users who prefer not to use the automated driver can manually navigate to a hero’s Proficiency tab and press `Alt+PrtScn`, then trigger a single-hero scan

from clipboard—giving full control over what the OCR processes.

5.3 Game Terms of Service Considerations

ProfTracker interacts with Marvel Rivals only through read-only screen capture. It does not:

- Read game memory
- Intercept game network traffic
- Modify game files
- Provide any mechanical advantage during gameplay

The Interception driver is used solely to navigate the hero roster during a session when the user is not actively playing. The application requires the user to be on the Heroes grid screen (not in a match) before scanning begins.

Fandom wiki data is retrieved via the public MediaWiki API with standard `User-Agent` identification (`ProfTracker/2.0.0 (educational project)`), respecting the rate limits and terms of the public API.

5.4 RAG Accuracy and Hallucination Risk

The RAG system is constrained to retrieval-augmented generation from the Fandom wiki. Groq llama-3.3-70b is prompted with the retrieved context and asked to answer only from that context (LlamaIndex's default `ResponseMode.COMPACT`). This reduces (but does not eliminate) hallucination risk.

Known failure modes: 1. **Partial index.** If only a subset of heroes are indexed, cross-hero aggregate queries produce incorrect counts. Mitigated by auto-sync, which rebuilds a full 51-hero index. 2. **Wiki inaccuracy.** If the Fandom wiki contains incorrect data, the model will faithfully reproduce the error. This is a limitation of using a community-edited source. 3. **Outdated index.** After a game patch that changes hero stats, the RAG index reflects the pre-patch values until the user re-runs wiki sync.

The disclaimer text in the HERO WIKI panel explicitly attributes answers to Groq AI and warns users that information may be incomplete.

5.5 Shared API Key Risk

The Groq API key is shipped with ProfTracker and shared across all users. This introduces several risks: - API costs for all user queries accrue to the developer - All users share the

same rate limit pool, meaning heavy usage by some users can degrade the experience for others - Key compromise or revocation requires issuing a new application release

The in-app disclaimer informs users that the RAG feature is powered by a shared Groq API key and that query volume may be subject to rate limiting.

5.6 Risk Register

The following table summarizes the project's tracked risks, their current likelihood and impact, implemented mitigations, and any remaining open actions.

ID	Risk	Likelihood	Impact	Status
R-001	Fandom wiki MediaWiki API changes break the avatar/ability scraper	Medium	High	Open — scraper targets stable <code>action=parse</code> / <code>action=query</code> endpoints rather than rendered HTML; scraper unit tests with fixture HTML not yet written
R-002	Groq free-tier rate limits block RAG queries	Medium	Medium	Mitigated — <code>QueryWorker</code> implements exponential backoff with jitter (1s, 2s, 4s, up to 3 retries); errors surface as in-chat messages
R-003	PyInstaller bundle omits RAG dependencies, breaking HERO WIKI in the frozen exe	Low	High	Open — <code>proftracker.spec</code> includes explicit <code>datas</code> entries; HuggingFace model downloads to user cache on first query; frozen-build smoke test in CI not yet implemented
R-004	OCR accuracy regresses after a Marvel Rivals patch changes the proficiency screen layout	High	Medium	Open — current mitigation is fixed fractional pixel regions + manual entry fallback; automated OCR regression fixture against saved screenshots not yet built
R-005	Hero roster becomes stale after a patch adds new heroes before the CDN sync runs	Medium	Low	Open — CDN refreshes daily; a “new heroes available” in-app banner when CDN roster count exceeds local count is not yet implemented
R-006	HuggingFace model download	Low	High	Open — setup notes document cache location; a <code>PROFTRACKER_DISABLE_RAG</code>

ID	Risk	Likelihood	Impact	Status
	fails in restricted network environments			flag to hide the HERO WIKI tab and skip model loading is not yet implemented
R-007	EasyOCR or torch version update breaks the capture pipeline	Low	Medium	Open — <code>pyproject.toml</code> specifies minimum versions; exact version pinning via <code>constraints.txt</code> in the PyInstaller CI step is not yet implemented
R-008	Client-side wiki sync hits Fandom rate limits at scale	Medium	Medium	Mitigated — CDN architecture (Section 2.5) moves all wiki scraping to a single daily EC2 job; end users never contact Fandom directly

Closed risks: R-Co1 (database wiped by app update — resolved in v1.0.6 by moving DB to `%APPDATA%`), R-Co2 (Interception driver crash — resolved by lazy-loading), R-Co3 (hardcoded roster — resolved by `heroes.json` CDN sync), R-Co4 (missing GROQ key crash — resolved by graceful in-chat error handling).

6. Lessons Learned and Future Work

6.1 What Worked Well

Two-document ingest strategy. The decision to index each hero page as both a compact stats chunk and a full-text chunk was the highest-impact design decision in the RAG pipeline. Before this change, HP and role queries were inconsistently answered because the relevant numeric values were diluted in long ability description chunks. After the change, single-hero stat queries achieved near-100% retrieval accuracy in evaluation.

QThread signal contract. Using a uniform `progress(int, int, str) / finished / error` signal interface across all worker types (scan, wiki sync, RAG query, etc.) made it

straightforward to wire every async operation to the same progress bar and log panel pattern. Adding a new background operation requires only writing a new `QThread` subclass; the UI wiring is identical for all of them.

Two-phase scan design. Separating navigation from OCR meant that Phase 2 (the slow EasyOCR processing) runs after the game window is released. Early single-phase designs held the game window for the duration of OCR, which took 30–60 seconds per hero and made the scan session interruptive. The current design keeps Phase 1 under 5 minutes for a full 51-hero scan.

Idempotent schema migration. Using `ALTER TABLE ... ADD COLUMN` with exception swallowing for schema migrations, rather than a versioned migration system, simplified the upgrade path significantly. Users who upgrade from an older version of ProfTracker find their data intact and any new columns automatically backfilled to defaults.

6.2 What Did Not Work

pytesseract for game UI. The initial OCR implementation used pytesseract as the primary engine. The italic, bold game font consistently confused pytesseract's character models, producing a high error rate on level numbers (the most critical field). The final implementation dropped pytesseract entirely in favor of EasyOCR, which handles the game's stylized font without preprocessing.

Cross-hero aggregate RAG queries. Queries like “How many vanguards are in the game?” require the model to count or enumerate across all indexed documents. LlamaIndex's default retrieval returns the top-3 most similar chunks, not a comprehensive set. For aggregate queries, the isolated stats aggregate document (`_insert_stats_aggregate`) partially addresses this—but the model still sometimes under-counts because the aggregate chunk may not be retrieved at rank 1. This remains an open limitation.

6.3 Cloud Data Distribution (AWS Architecture)

Scraping the Fandom wiki directly from each user's machine creates a rate-limiting risk: as the user base grows, concurrent sync operations from many machines could trigger Fandom's rate limiting, causing syncs to fail or produce incomplete icon sets. ProfTracker addresses this with a fully implemented cloud-hosted data distribution layer.

Architecture (three AWS services):

1. **EC2 (t3.micro):** A scheduled Python process (`server_sync.py`) runs the full wiki scrape once daily, writing the output (`heroes.json` , `hero_data/*.json` , `Icons/`) to an S3 bucket via boto3. This eliminates the Fandom scraping load from end users entirely.
2. **S3 (Standard storage):** The output files are stored in a public-read S3 bucket (`proftracker-wiki-data-clayhtryon.s3.amazonaws.com`). At roughly 300 MB for all icons and <1 MB for JSON data, monthly storage cost is approximately \$0.07. The S3 bucket serves as the canonical source of truth for the hero roster.
3. **Amazon EventBridge (Scheduler):** A scheduled rule triggers the EC2 scraper daily at 06:00 UTC. EventBridge is effectively free at this invocation rate.

Client integration: `CDN_BASE` is hardcoded in `avatar_sync.py` to the S3 bucket URL, so all clients always sync from CDN without requiring any environment configuration. This reduces sync time for end users from 2–5 minutes (sequential wiki API calls) to under 30 seconds (parallel S3 downloads) and ensures all users work from the same consistent daily dataset.

6.4 Known Limitations

The following limitations are present in the current release and are documented for transparency:

- **Windows-only.** The Interception kernel driver and `win32gui` APIs used by the Auto Scan feature are Windows-specific. The application will install and run on other platforms, but the Auto Scan tab is non-functional outside of Windows 10/11.
- **English-only.** The OCR character allowlists, UI strings, and wiki data pipeline are all English. Non-English game clients or localized wiki pages are not supported.
- **1920×1080 resolution optimized.** The fractional pixel coordinate approach scales correctly for any 16:9 aspect ratio, but the OCR regions have been validated only at 1080p windowed mode. Ultra-wide or non-standard resolutions may require recalibration.
- **CDN dependency at first run.** A fresh install requires an internet connection to download hero data from S3. The built-in roster fallback provides a degraded experience (no icons or ability data) if the CDN is unavailable at setup time.

- **No multi-device sync.** Proficiency data is stored in a local SQLite database and does not sync between machines.

6.5 Future Work

The following items are planned next steps, several of which directly address open risks from the risk register (Section 5.6):

OCR regression test fixture (R-004). The `.temp/` directory accumulates OCR debug screenshots automatically during every scan. A CI step could hash these images and flag when OCR output for a known screenshot changes after a code update, providing an automated regression signal for proficiency screen layout changes without requiring a live game session.

Scraper unit tests with fixture HTML (R-001). The wiki scraper targets stable MediaWiki API endpoints, but API response structure can still change. Capturing fixture responses and running the parser against them in CI would catch regressions before users see sync failures.

Frozen-build smoke test in CI (R-003). The release workflow builds with PyInstaller but does not verify that the RAG features work in the frozen executable. A post-build smoke test that launches the exe and runs a test query would confirm the HuggingFace and LlamaIndex dependencies are correctly bundled.

PROFTRACKER_DISABLE_RAG flag (R-006). Users in restricted network environments (corporate proxies, air-gapped machines) cannot download the HuggingFace embedding model. An environment flag to hide the HERO WIKI tab and skip model loading entirely would allow the rest of the app to function normally in these environments.

New-hero available banner (R-005). When the CDN's hero roster count exceeds the local count, an in-app banner could prompt the user to re-sync, surfacing new heroes without requiring manual awareness of game patches.

Multi-resolution OCR calibration. Automated detection of the game window's resolution would allow the fractional pixel regions to be validated and adjusted per display, expanding the addressable user base beyond the current 1080p optimization.

Hero skin library. Each hero detail page could include a dedicated Skins tab displaying the full cosmetic roster — skin name, rarity, source, cost, and preview image. The cosmetics scraper (`cosmetics_scraper.py`) is implemented; the UI tab is the remaining deliverable.

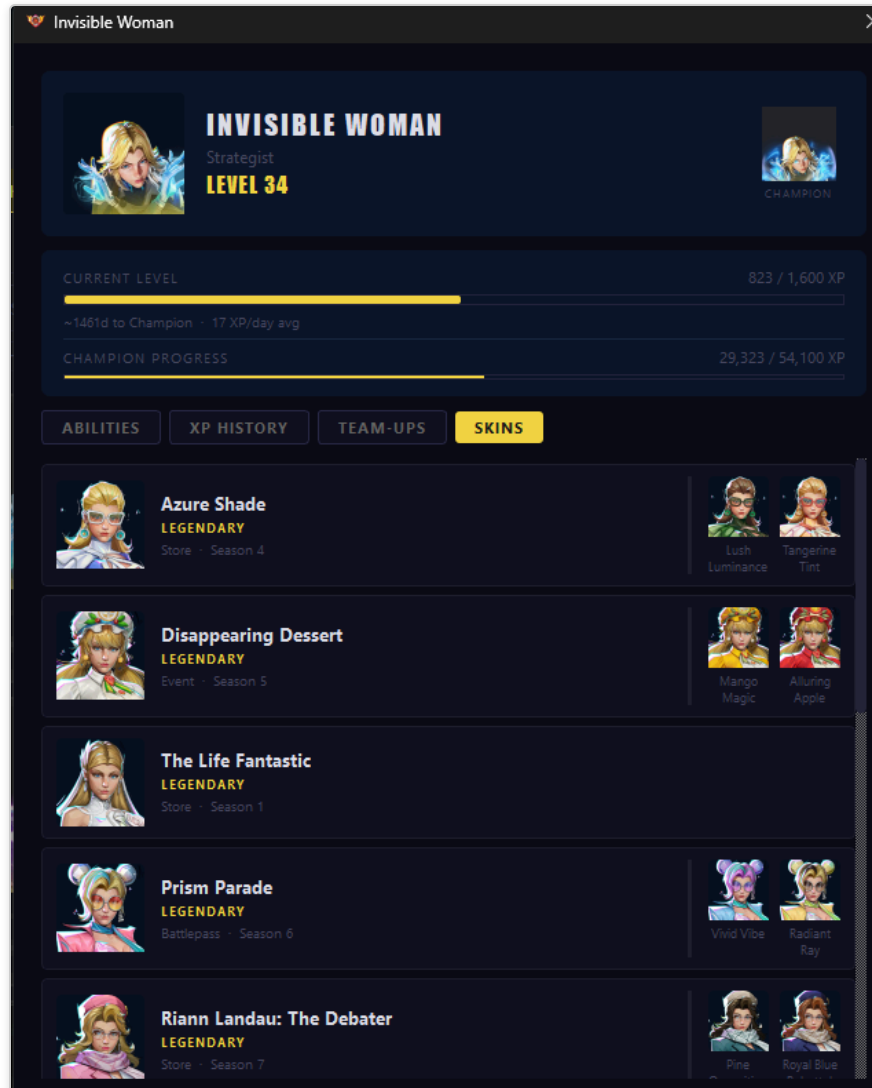


Figure 6: Hero Skins tab (Invisible Woman) — cosmetic library showing skin names, rarity tiers (Legendary), acquisition methods (Store / Event / Battlepass), and colour variant thumbnails. This tab is already implemented; the UI panel is the remaining deliverable.

Summary

ProfTracker demonstrates the integration of multiple AI and systems engineering components into a functional, user-facing desktop application. The core technical contributions are:

- A two-phase OCR scan pipeline using EasyOCR with XP-table cross-validation that achieves reliable accuracy on stylized game UI text
- A LlamaIndex RAG system with a two-document ingest strategy that isolates game stats from lore text to improve retrieval precision
- A MediaWiki API-based wiki sync pipeline, offloaded to a daily AWS EC2 job, that provides consistent hero roster data to all clients via S3
- A PyQt6 desktop architecture using QThread workers with a uniform signal contract across all async operations

The system processes 51 heroes in a single automated scan, indexes the full game wiki for conversational queries, and stores all proficiency data locally with no ongoing cloud dependency for the end user beyond the initial setup sync.